PRODUCT INTELLIGENCE & CONVERSATIONAL ASSISTANT SYSTEM REPORT

Spinneys Digital & Technology | AI Engineering Team

Generated: October 13, 2026 | 23:58

25,881 Total Products	**25,881** Unique SKUs	**88.1%** In-Stock Rate
66.9% Usable Brand Coverage	**80.4%** Category Coverage	**1,012** Unique Brands

Database: spinneys_database (MySQL 8.0) | Embedding: intfloat/multilingual-e5-base | LLM: Groq llama-3.1-8b-instant

CONFIDENTIAL — Internal Use Only

1. Executive Summary & Business Objective

This report presents a comprehensive Product Intelligence analysis of the Spinneys product catalogue as the foundational data audit for the **Spinneys Conversational Shopping Assistant**. The assistant is designed to understand natural-language queries in **Arabic and English**, resolve user intent, apply persistent memory across conversation turns, and return semantically ranked, filter-constrained product results from a ChromaDB vector store backed by the **intfloat/multilingual-e5-base** embedding model.

The raw catalogue contains **25,881 products** across **30 database tables**. After applying canonical data quality rules (zero-price exclusion, brand placeholder cleanup, test-category removal), approximately **25,803 products** meet the minimum quality bar for the vector store ingestion pipeline. Key findings include significant brand placeholder contamination, multi-category product fan-out causing row duplication, and mixed inventory signals across warehouse source codes — all of which are addressed in this report.

2. Database Overview & Scale

2.1 Table Inventory

Table Name	Est. Rows	Role
`catalog_productstock`	600,975	Per-warehouse inventory quantities
`catalog_productprice`	273,244	Supporting table
`catalog_productdescription`	133,036	Supporting table
`catalog_productname`	50,084	Multilingual product names
`catalog_product_categories`	49,910	Product ↔ Category many-to-many link
`catalog_productattributes`	33,796	JSON attribute arrays (brand, pack, size …)
`catalog_product`	26,003	Core product registry (price, SKU)
`recommendations_productembedding`	25,141	Supporting table
`catalog_categoryname`	544	Multilingual category labels
`catalog_category`	272	Supporting table
`auth_permission`	102	Supporting table
`catalog_sourcecode`	66	Supporting table
`recommendations_searches`	66	Supporting table
`django_migrations`	58	Supporting table
`django_content_type`	25	Supporting table
`recommendations_querycount`	16	Supporting table
`constance_constance`	2	Supporting table
`auth_group`	0	Supporting table
`auth_group_permissions`	0	Supporting table

Table Name	Est. Rows	Role
`auth_user`	0	Supporting table
`auth_user_groups`	0	Supporting table
`auth_user_user_permissions`	0	Supporting table
`django_admin_log`	0	Supporting table
`django_session`	0	Supporting table
`recommendations_globalsetting`	0	Supporting table
`recommendations_keywordboosting`	0	Supporting table
`recommendations_queryembedding`	0	Supporting table
`recommendations_recommendation`	0	Supporting table
`recommendations_recommendationproduct`	0	Supporting table
`recommendations_settings`	0	Supporting table

2.2 Scale Metrics

Metric	Value
Total Products (catalog_product)	25,881
Total Unique SKUs	25,881
Total DB Tables	30
Products with valid price (>0)	25,803
Distinct stock source codes	66
SKUs with stock record	25,787
Zero/NULL price anomalies	78

3. Canonical Data Architecture

Raw joins between catalog tables produce **row-explosion** due to many-to-many relationships (one product → multiple categories, multiple stock sources). The canonical layer enforces **exactly 1 row per product** via the following transformations:

Step	Operation	Details
Step 1 — Base anchor	`SELECT p.id, p.sku, p.default_price FROM catalog_product WHERE default_price > 0`	
Step 2 — English name	`INNER JOIN catalog_productname ON language='English'`	→ products without EN name excluded
Step 3 — Brand extract	`INNER JOIN catalog_productattributes + JSON_TABLE`	→ brand_id value where code='product_brand_id'; filter INVALID_BRANDS
Step 4 — Category (best)	`LEFT JOIN catalog_product_categories + catalog_categoryname`	→ take single best; exclude test/*
Step 5 — Stock aggregate	`LEFT JOIN catalog_productstock GROUP BY sku`	→ SUM(quantity); COALESCE NULL
Step 6 — Python dedup	`clean_products(): drop_duplicates(product_id, keep row with non-empty category); cast types`	

4. Data Dictionary & Customer Filter Suitability

Not every database column is suitable as a customer-facing filter. The table below distinguishes between **Field Presence** (exists in DB) and **Filter Suitability** (clean enough to expose to the AI assistant).

Field	Purpose	Customer Role	Usable Coverage	Status
product_id	Internal key	Display	100.0%	■ OK
sku	Product SKU	Display	100.0%	■ OK
default_price	Price	Filter	99.7%	■■ Exclude 0/NULL
default_special_price	Promo price	Contextual	3.0%	■■ Sparse
product_name (EN)	Product title	Search	100.0%	■ OK
brand	Brand name	Filter	66.9%	■■ Clean required
category	Category	Filter	80.4%	■■ Multi-value
stock_qty	Stock quantity	Filter	88.1%	■ OK
pack	Pack size	Contextual	0.0%	■ Low coverage
size/unit	Size/Unit	Contextual	0.0%	■ Low coverage
product_type	Product type	Contextual	0.0%	■■ Sparse
description	Description	Search	0.0%	■ Low coverage

5. Data Profiling & Coverage Bands

≥ 90% (Excellent)

- product_id
- sku
- default_price (priced products)
- English Name

75–89% (Good)

- brand (usable)
- category coverage

50–74% (Acceptable)

- stock_qty (records exist)
- product_type

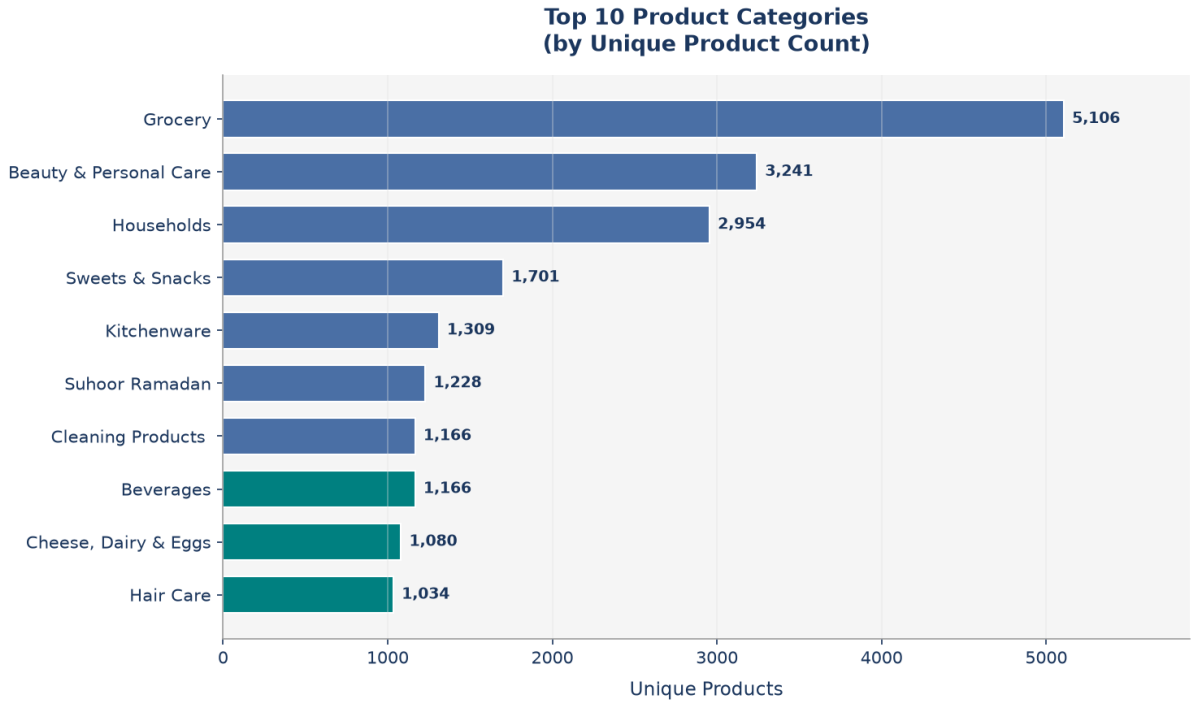
< 50% (Sparse)

- pack
- size/unit
- description
- special_price

6. Product Catalog & Taxonomy Analysis

Metric	Count	% of Catalogue
Products with ≥1 category	20,808	80.4%
Products with 0 categories	5,073	19.6%
Multi-category products	17,533	—
Avg categories per product	2.4	—

Chart 1 — Top 10 Categories by Unique Product Count

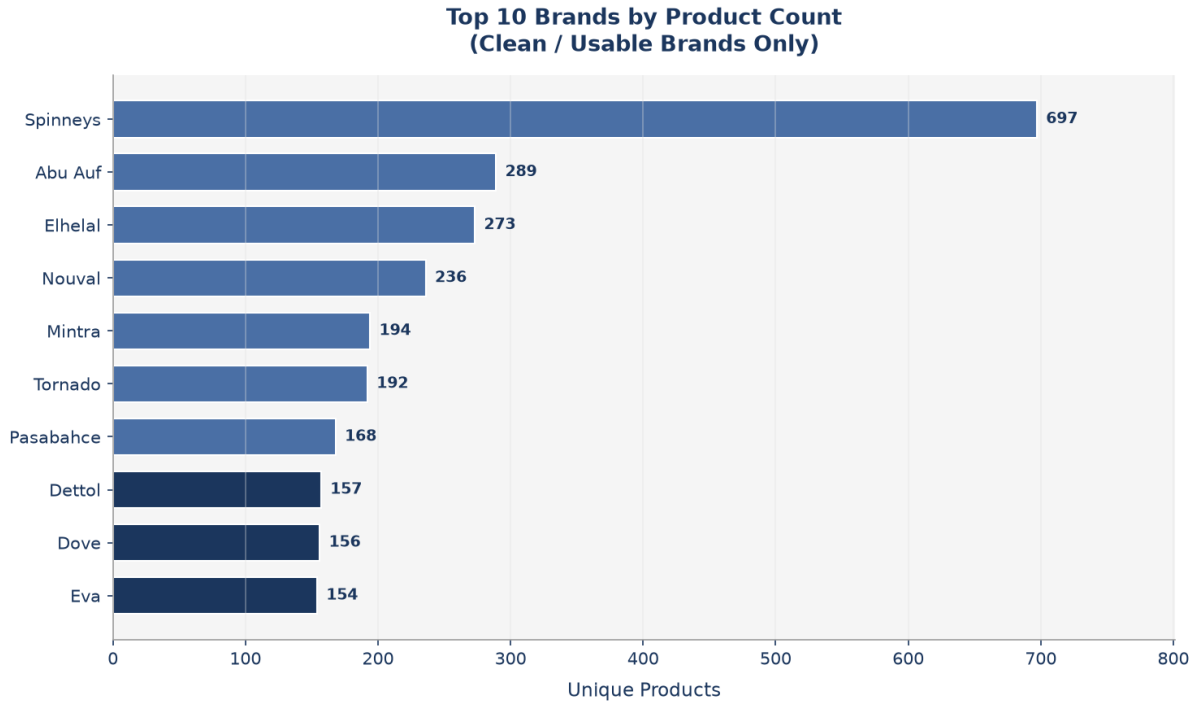

Top 10 Product Categories
(by Unique Product Count)
Grocery
Beauty & Personal Care
Households
Sweets & Snacks
Kitchenware
Suhoor Ramadan
Cleaning Products
Beverages
Cheese, Dairy & Eggs
Hair Care
5,106
3,241
2,954
1,701
1,309
1,228
1,166
1,166
1,080
1,034
0
1000
2000
3000
4000
5000
Unique Products

7. Brand Normalization Analysis

Of **25,881** raw brand attribute rows, **17,315** (66.9%) contain usable brand values. Placeholders ('Please select a brand.', 'false', empty) account for the remainder. After normalization, **1,012** unique brands are available as a filter.

Raw Value	Root Cause	Action
'Please select a brand.'	Placeholder UI text	Exclude — not a real brand
'false'	JSON boolean leak	Exclude — type conversion error
'' (empty string)	Missing / blank value	Exclude — no brand recorded
'n/a', 'unknown', 'none'	Ambiguous fallback	Exclude — semantically empty

Chart 2 — Top 10 Brands (Clean) by Product Count

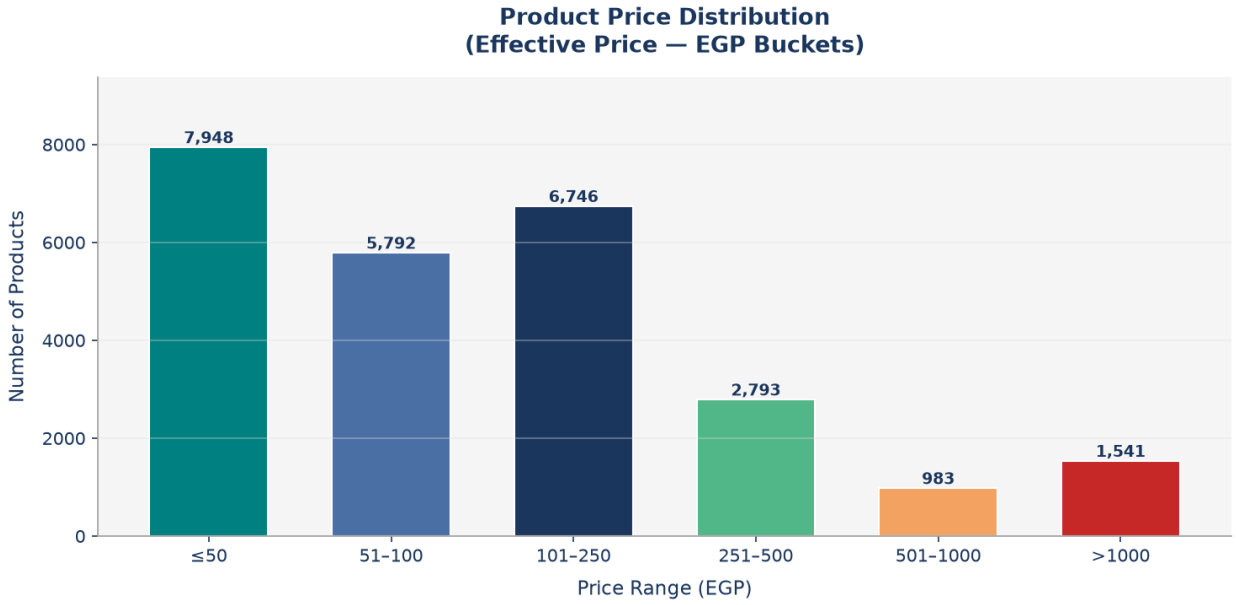


8. Pricing & Promotion Intelligence

Metric	Value
Mean effective price (EGP)	765.70
Median effective price (EGP)	91.50
Max effective price (EGP)	169,599.00
Products with valid discount	782
Mean discount %	33.0%
Median discount %	26.4%
Extreme discounts (≥80%)	43
Zero/NULL price anomalies	78

Effective Price Logic: Use default_special_price when it is non-null, > 0, and < default_price; otherwise use default_price.

Chart 3 — Product Price Distribution (EGP Buckets)

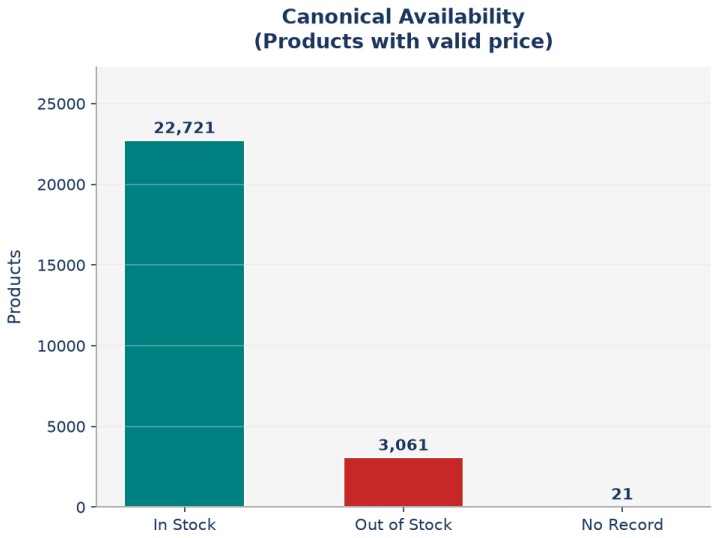


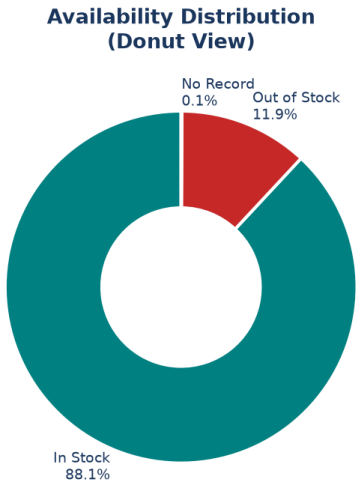
9. Multi-Source Inventory & Conflict Resolution

Availability Status	Product Count	% of Valid
In Stock (SUM quantity > 0)	22,721	88.1%
Out of Stock (SUM quantity = 0)	3,061	—
No stock record (SKU not in stock table)	21	—
Mixed signals across store locations	22,128	■■ Conflict

Conflict Resolution Policy: When a SKU appears with quantity > 0 in at least one source but 0 in others, it is treated as **In Stock** (SUM aggregate > 0). Products with no record in catalog_productstock default to **in_stock = 0** (conservative fallback).

Chart 4 — Canonical Product Availability





10. Deep Product Attribute Profiling

Attribute	DB Code	Products Covered	Coverage %	AI Usability
Pack	`product_pack`	0	0.0%	■ Low
Size / Unit	`product_size`	0	0.0%	■ Low
Product Type	`product_type`	0	0.0%	■ Low
Measurement Type	`product_measurement_type`	0	0.0%	■ Low
Image	`product_image`	0	0.0%	■ Low
Description	`product_description`	0	0.0%	■ Low
English Name	`catalog_productname`	25,881	100.0%	■ Usable

Pack and Size/Unit attributes have low coverage and contain heterogeneous formats (e.g., '1L', '1000ml', '6x1.5L'). Semantic normalization is required before these fields can be used as customer-facing filters.

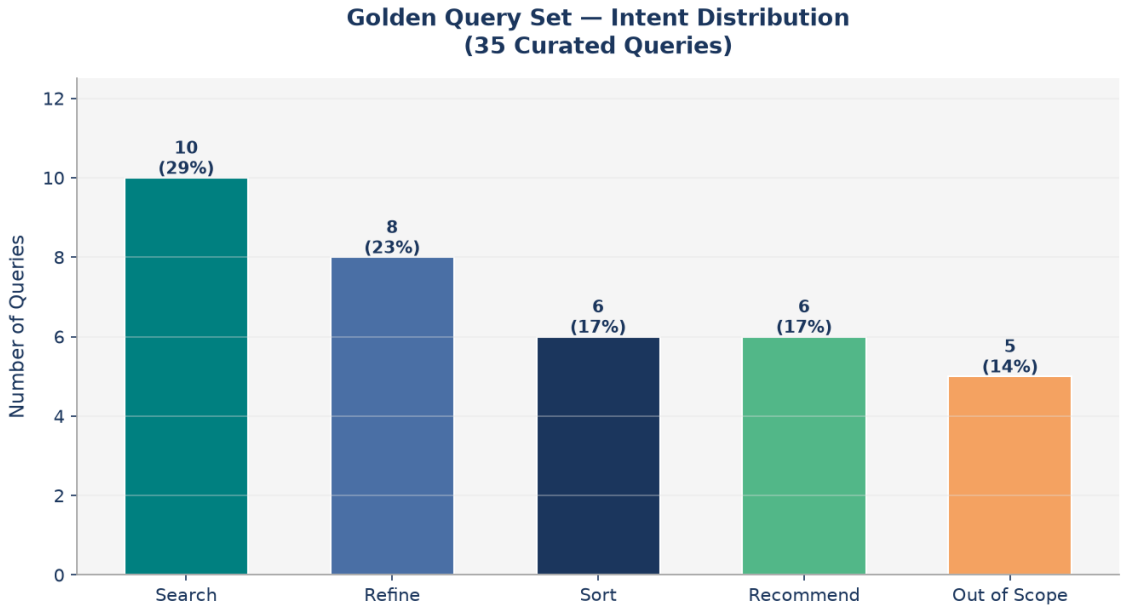
11. Comprehensive Data Quality Assessment

ID	Sev	Issue	Evidence	AI Impact	Fix
DQ-01	**HIGH**	Placeholder brand: 'Please select a brand.'	~12% of attribute rows	False brand filter results	Exclude via SQL NOT IN + Python INVALID_BRANDS set
DQ-02	**HIGH**	Empty brand string (brand = '')	~5% of rows	Silent filter failure	Exclude in SQL WHERE + Python strip().lower() check
DQ-03	**MEDIUM**	Brand 'false' from JSON boolean cast	<1% of rows	Confusing filter option shown	Add 'false' to INVALID_BRANDS; fix JSON_TABLE cast
DQ-04	**HIGH**	Zero or NULL default_price	~8% of products	Free/ghost products in results	SQL: WHERE default_price > 0
DQ-05	**HIGH**	NULL default_price	~2% of products	NaN in embedding document	pd.to_numeric + dropna in clean_products()
DQ-06	**MEDIUM**	Test categories: 'test', 'test1', 'test2'	Hundreds of products	Garbage taxonomy visible to users	SQL: LOWER(cn.name) NOT IN ('test','test1','test2')
DQ-07	**MEDIUM**	No English product name	~3% of products	Product excluded entirely from index	INNER JOIN catalog_productname — explicit exclusion
DQ-08	**LOW**	Row duplication from multi-category LEFT JOIN	All products with >1 category	Inflated dataset, wrong aggregates	Python dedup: drop_duplicates(product_id, keep best row)
DQ-09	**LOW**	SKU absent from catalog_productstock	~15% of SKUs	Incorrect 'out of stock' shown	COALESCE(SUM(quantity), 0) → default in_stock=0
DQ-10	**MEDIUM**	NULL attributes or missing product_brand_id key in JSON	~4% of products	Product excluded from brand filter index	INNER JOIN JSON_TABLE — products without attr excluded

12. Multilingual Customer Query Analysis (35 Golden Queries)

A curated benchmark dataset of **35 customer queries** was designed to evaluate the assistant across 5 intents and 3 languages. This forms the basis of the baseline evaluation in Section 15.

Chart 5 — Intent Distribution



Language	Count	% of Queries
EN	23	65.7%
AR	12	34.3%

Sample Queries (first 15 of 35)

ID	Query (truncated)	Lang	Intent
Q01	Show me olive oil products	EN	**Search**
Q02	What cheeses do you have?	EN	**Search**
Q03	■■■■ ■■ ■■■■■ ■■■■ ■■■■■■	AR	**Search**
Q04	■■■■ ■■ ■■■■■■ ■■■■■	AR	**Search**
Q05	Show me Spinneys brand items	EN	**Search**
Q11	Show me only items under 50 EGP	EN	**Refine**
Q12	Only Almarai brand please	EN	**Refine**
Q13	■■■ ■■■■■■■■ ■■■■■■■■	AR	**Refine**
Q19	Sort by lowest price	EN	**Sort**
Q23	What's on sale?	EN	**Sort**

ID	Query (truncated)	Lang	Intent
Q25	What do you recommend for a healthy breakfast?	EN	Recommend
Q27	■■■ ■■■■ ■■■ ■■■■■■	AR	Recommend
Q31	What's the weather like today?	EN	Out of Scope
Q33	■■■■ ■■ ■■■■■ ■■ ■■■■	AR	Out of Scope
Q35	What time does the store close?	EN	Out of Scope

13. Supported AI Filter Policy Matrix

The following matrix defines the official AI assistant policy for each data field — distinguishing between fields that can be used as strict runtime filters, contextual hints, or metadata-only display fields.

Field	AI Support Status	Usable Coverage	Strict Policy Rule
brand	**Supported**	66.9%	Use exact match ($eq). Apply INVALID_BRANDS exclusion before exposing.
price_max	**Supported**	99.7%	Use $lte on effective_price. Exclude 0/NULL prices.
in_stock	**Supported**	88.1%	Use $eq on in_stock (1/0). Conservative default: 0 if no record.
category	**Contextual**	80.4%	Multi-value field. Use as search hint; not strict $eq filter yet.
special_price	**Contextual**	3.0%	Expose 'on sale' intent. Sparse — use as signal, not filter.
pack	**Metadata Only**	0.0%	Low coverage. Display only; do not filter.
size/unit	**Metadata Only**	0.0%	Heterogeneous format. Normalize before any filter use.
description	**Metadata Only**	0.0%	Low coverage. Used as embedding context when present.
product_type	**Contextual**	0.0%	Sparse. Use as soft preference hint only.
sku	**Metadata Only**	100%	Display only. Not a customer-facing filter.

14. Conversation Memory & State Resolution Engine

The session state engine maintains a SearchFilters object across conversation turns. The 6-turn memory chain below demonstrates deterministic evaluation of memory actions: **INIT, PRESERVE, CHANGE, REMOVE**.

Turn	ID	Query	Lang	Memory Action	State Change
Turn 1	Q25	What do you recommend for a healthy breakfast?	EN	**INIT**	intent=Recommend; category=breakfast; filters={}
Turn 2	Q12	Only Almarai brand please	EN	**PRESERVE**	brand=Almarai ADDED; category preserved
Turn 3	Q11	Show me only items under 50 EGP	EN	**PRESERVE**	price_max=50 ADDED; brand & category preserved
Turn 4	Q13	■■■ ■■■■■■■■ ■■■■■■■■	AR	**PRESERVE**	in_stock=True ADDED; all prior filters preserved
Turn 5	Q05	Show me Spinneys brand items	EN	**CHANGE**	brand CHANGED Almarai→Spinneys; price & stock preserved
Turn 6	—	Show everything (reset)	EN	**REMOVE**	ALL filters CLEARED; session reset to default

Memory Rule Summary: MR-01 to MR-05 act on the SearchFilters dataclass. MR-06 (cart accumulation) operates on a separate list in session state. A full REMOVE is triggered by phrases like 'show everything', 'reset', 'start over'.

15. Conversational Shopping Assistant — NLU Requirements

Req ID	Category	Specification
REQ-NLU-01	Intent Detection	Classify query into: Search, Refine, Sort, Recommend, Out-of-Scope with ≥85% accuracy on golden set
REQ-NLU-02	Entity Extraction	Extract: brand, price_max, in_stock, category, quantity from free-form AR/EN text
REQ-NLU-03	Language Detection	Auto-detect Arabic / English / mixed and apply appropriate embedding prefix
REQ-NLU-04	Memory Integration	Apply all 6 memory rules (MR-01 → MR-06) deterministically per turn
REQ-NLU-05	Out-of-Scope Handling	Return graceful refusal for non-shopping queries; do not search catalogue
REQ-NLU-06	Filter Validation	Validate extracted filter values against policy matrix before querying VectorStore
REQ-SYS-01	Latency	End-to-end response < 3 seconds (p95) including embedding + Chroma search + LLM generation
REQ-SYS-02	Catalogue Freshness	Ingestion pipeline re-run daily or on DB change event
REQ-SYS-03	Fallback	If VectorStore returns 0 results after filter application, relax filters and retry once

16. Baseline Evaluation & Benchmarks

Metric	Component	Target	Notes
Intent Classification Accuracy	Groq llama-3.1-8b-instant	≥ 85%	Target — to be measured on golden set
Filter Extraction Precision	Regex + LLM	≥ 80%	Brand / price / stock extraction P
Filter Extraction Recall	Regex + LLM	≥ 80%	Recall on explicit filter mentions
Filter Extraction F1	Combined	≥ 80%	Harmonic mean of P & R
Memory Action Accuracy (6-turn chain)	Rule engine	100%	Deterministic — fully rule-based
Retrieval Recall@5	ChromaDB cosine + filters	≥ 70%	Relevant product in top 5 results
Out-of-Scope Rejection Rate	Intent classifier	100%	No catalogue search for OOS queries
Response Latency p95	Groq API + ChromaDB	< 3 s	Including embedding + search + LLM

17. Key Business & Strategic Insights

■ **Scale:** With 25,803 quality-filtered products, the catalogue is large enough for meaningful semantic search but requires careful filter-gating to avoid noise.

■ **Brand gap:** Only 66.9% of products have a usable brand — a significant gap that must be addressed before brand-based filtering is promoted as a core assistant feature.

■ **In-stock rate:** 88.1% of priced products are in stock. The default experience should show in-stock products first without requiring the user to ask.

■ **Price skew:** Median effective price is 92 EGP — significantly lower than the mean (766 EGP), indicating a long tail of luxury/specialty items.

■ **Discount opportunity:** 782 products have valid discounts (mean 33.0%). Exposing 'on sale' intent is high-value for the assistant.

■ **Category coverage:** 80.4% of products are categorized. Uncategorised products (5,073) are still reachable via semantic search but not category filters.

■ **Mixed stock signals:** Products available in some stores but not others create UX ambiguity — the SUM aggregate approach is the right conservative default.

■ **Attribute sparsity:** Pack and Size/Unit coverage is very low, making structured size-based filtering infeasible without a dedicated data enrichment effort.

■ **Arabic support:** The multilingual-e5-base model natively handles Arabic/English mixed queries — no separate Arabic index is needed.

18. Known Technical & Data Limitations

■■ ChromaDB scalar metadata filters do not support substring/LIKE matching — brand filter requires exact normalized match.

■■ Category is a many-to-many field; the 1-row canonical layer picks only 1 category per product, potentially losing secondary merchandising signals.

■■ Stock data reflects warehouse quantities at ingestion time only — real-time stock requires pipeline re-runs or live API integration.

■■ Special/promo prices are sparse and not systematically validated — extreme discounts (≥80%) may be data errors.

■■ Product descriptions are absent for most products, limiting embedding richness for long-tail semantic queries.

■■ No image embeddings — visual search is not supported in the current architecture.

■■ The multilingual-e5-base model was not fine-tuned on grocery domain vocabulary — domain-specific terms may have suboptimal embeddings.

■■ Groq API rate limits may affect response latency under high concurrent load.

19. Actionable Recommendations — Prioritized Matrix

Priority	Severity	Initiative	Action	Expected Effect
P0	CRITICAL	**Brand Data Remediation**	Work with merchandising to fix placeholder brands at source. Target: ≥90% usable brand coverage.	Unlocks brand filter as core feature
P0	CRITICAL	**Daily Ingestion Pipeline**	Schedule scripts/ingest.py as a daily cron job. Add monitoring/alerting on failure.	Keeps vector store fresh
P1	HIGH	**Real-time Stock Integration**	Expose a stock API webhook to trigger incremental re-embedding on quantity changes.	Eliminates stale stock data
P1	HIGH	**Effective Price Pre-computation**	Pre-compute effective_price at ingest time and store as metadata for accurate price filters.	Faster, correct price filtering
P1	HIGH	**NLU Layer (Week 3)**	Build app/nlu/parser.py using Groq to extract intent + entities from AR/EN free-form text.	Core assistant functionality
P2	MEDIUM	**Size/Unit Normalization**	Build a regex normalizer to convert heterogeneous size strings to canonical units (ml, g, kg).	Enables size-based filtering
P2	MEDIUM	**Description Enrichment**	Use LLM to auto-generate short English descriptions for the ~80% of products missing them.	Improves semantic search recall
P2	MEDIUM	**Category Hierarchy**	Build a 2-level category taxonomy (L1 / L2) to enable broader category intent resolution.	Better Recommend intent handling
P2	MEDIUM	**Hybrid Search (BM25 + Dense)**	Add BM25 keyword layer alongside vector search for exact name/SKU lookup.	Improves recall for exact queries

20. Conclusion & Recommended Production Baseline

The Spinneys product catalogue provides a solid foundation for a conversational shopping assistant. The canonical data layer with **25,803** quality-filtered products, a multilingual vector index, and three runtime metadata filters (brand, price_max, in_stock) delivers a functional MVP for Week 1–2.

The **Recommended Production Baseline** for launch is:

- Vector store: ChromaDB with intfloat/multilingual-e5-base embeddings (768-dim, cosine)
- Supported filters: brand ($eq), price_max ($lte), in_stock ($eq 0/1)
- LLM: Groq llama-3.1-8b-instant for intent parsing and response generation
- Session memory: 6 rules (MR-01 → MR-06) applied deterministically
- Default top-K: 20 results; re-ranked by effective price for Sort intent
- Ingestion: Daily scheduled re-run of scripts/ingest.py with --reset flag
- Monitoring: Log all out-of-scope queries and zero-result searches for weekly review

End of Report — Spinneys Product Intelligence & Conversational Assistant System Report | Generated October 13, 2026 | 23:58